SMART FLIGHT CHECK-IN KIOSK

GROUP 35

GROUP MEMBER

NAME	*QM ID*	*BUPT ID*
Qian Ma	**190898742**	**2019213176**
Wenxin Ning	**190898845**	**2019213177**
Qianrui Teng	**190899222**	**2019213133**
Quan Yuan	**190900056**	**2019213082**
Zitang Zhou	**190900539**	**2019213178**
Sijie Zou	**190900621**	**2019213171**

The total page of report is **15**. (exclude the contents and appendix)

CONTENTS

1. Introduction 3
2. Agile Project Management 3
   - 2.1 Initial Phase 3
     - 2.1.1 Project Planning 3
     - 2.1.2 Project Schedule 4
   - 2.2 Iterative Cycle 4
     - 2.2.1 Assess (Software Backlog) 4
     - 2.2.2 Select (Requirements) 4
     - 2.2.3 Develop (Java Coding and Debug) 4
     - 2.2.4 Review and Testing (Debug and Finish) 5
   - 2.3 Final Closure Phase 5
   - 2.4 Risk Management 5
   - 2.5 Quality Management 5
3. Requirement 5
   - 3.1Techniques for Finding Requirements 5
     - 3.1.1 Background Reading 5
     - 3.1.2 Questionnaire Survey 6
     - 3.1.3 Observation 6
   - 3.2 Prototype User Interface 6
   - 3.3 Detail Description of Requirements 6
     - 3.3.1 Functional Requirements 6
     - 3.3.2 Non-Functional Requirements 7
   - 3.4 Priorities and Estimation of Stories 7
     - 3.4.1 Priorities of the User Stories 7
     - 3.4.2 Product Backlog Evolution 7
     - 3.4.3 Improvement of Previous Backlog 7
4. Analysis 8
   - 4.1 Identify Entity, Boundary and Control Classes 8
     - 4.1.1 Control Classes 8
     - 4.1.2 Boundary Classes 8
     - 4.1.3 Entity Classes 9
   - 4.2 Conceptual Class Diagram 9
5. Design 10
   - 5.1 Reusability and Adaptability 10
   - 5.2 Design Principles 10
     - 5.2.1 SRP(Single Liability Principle) 10
     - 5.2.2 OCP (Open-Closed Principle) 11
     - 5.2.3 DRY(Don't Repeat Yourself) 11
     - 5.2.4 DIP (Dependency Inversion Principle) 11
     - 5.2.5 ISP (Interface Segregation Principle) 12
     - 5.2.6 LSP (Liskov Substitution Principle) 12
6. Implementation 12

6.1 Configuration Management 12
6.2 Iteration Plan 13
6.3 Implement Strategy 14
6.4 Integration of Build Plan 14
7. Testing 14
7.1 Testing Strategy 14
7.2 Testing Techniques 14
7.3 Test Case 15
7.4 Test Matrix 16
7.5 TDD 17
8. Conclusion 17

Appendix 18
1. Prototype User Interface 18
2. Priorities of the User Stories 19
3. Improvement of Previous Backlog 20
4. Test Case and Test Matric 20
5. Main Screenshots of the System 24
6. Participation and Achievement 32
7. References 33

1. Introduction

We have designed an autonomous check-in system, which is suitable for various airlines. Its main functions include: Enter the system in three ways, select the free seat and provide special seat selection, select a variety of different types of meals, record and calculate the extra charge, payment, confirm the information and print the boarding pass, luggage ticket, background storage user check-in status and so on. Before designing the function of the system, we analyzed the user's demand, and mined out more functions in line with the actual demand; Secondly, we drew the prototype of the system and presented the ideas in the form of visualization, so that users can have a clearer understanding of our system and its functions. Two people worked together to complete a task in the form of pair programming. Using the agile management method, we divided the process into five iterations. In the first phase, we realized basic functions such as a login system, basic seat selection, meal selection, etc. More complex features were gradually refined in iterations 2 and 3, and system beautification, system testing, and stability enhancements began in phases 4 and 5. At each stage, we modify the backlog and the user needs to find more practical features based on user feedback and experience.

2. Agile Project Management

We used Balsamiq Mockplus tool and Evolutionary development model in the Agile management to achieve our software. In general, we went through three stages in developing this software.

a. Initial phase (Project planning, Task allocation, and Architectural design).

b. Iterative cycle (Assess, Design, Development, Review, and Testing).

c. Final closure phase.

2.1 Initial Phase

2.1.1 Project Planning

At the beginning, according to the clear requirements in Handout and the materials provided by the lecturer in class, we determined the work to be completed in the initial stage of the project and made a series of constraints (for example, the time of handing in the first version of materials within the group, the uniformity of software use, etc.). Then we referenced requirements, wrote user stories, and used Balsamiq Mockplus to design our prototype, and used XMind to draw UML diagram, and so on. We defined

our milestones and deliverables for the nearest deadline.

Meanwhile, we set a reasonable schedule while completing the initial stages of software development, after that we did the initial coding and adjusted the solution and constraints as needed.

2.1.2 Project Schedule

We divided the entire software into many small pieces, reducing the correlation between each part, and used evolutionary development model in the Agile management to achieve our software. Every once in a while, we will hold a group meeting to define the next deadline and tasks to be completed in the next stage, and adjust our planning.

a. Conceive of Software Structure.
b. Logo and the interface of three ways of checking-in.
c. Three relative interfaces.
d. Interface if successful retrieve and payment interface.
e. Interfaces of seat selection and payment for special seats.
f. Interfaces of meal selection and payment for special meals.
g. Interface of confirmation and print.

2.2 Iterative Cycle

2.2.1 Assess (Software Backlog)

First of all, we looked for check-in systems at home and abroad, and finally decided to code and make GUI design according to the requirements of domestic software pages and requirements in the Handout.

2.2.2 Select (Requirements)

In agile development, to acquire an efficient implementation of our system, we need to make our requirements as accurately as possible. We wrote user stories from the user and airport staff two points of view. So, at the beginning of each iteration, we always held a group meeting to select the functions implemented from the Product Backlog.

2.2.3 Develop (Java Coding and Debug)

We wrote the code in Java and discussed the structure and names of several common functions, and chose web-based architecture to accomplish the other parts.

2.2.4 Review and Testing (Debug and Finish)

After completing all the stories, integrate and focus on testing. When there is a bug, the responsible person will debug and rectify it. When there is a function that needs to be expanded or adjusted, we will record the part that needs to be adjusted in the backlog and implement it uniformly. We not only ensure the correct operation of the check-in system, but also optimize the visual experience of GUI pages, like providing photos of payment software such as Alipay, and price matching of food selection system. At the same time, we also noticed the impact of Windows and Mac on how our code works.

2.3 Final Closure Phase

After 5 iterations which cost about 2 months, we finished our Smart Flight Check-in Kiosk which can meet all requirements of users and airport staff. Then the project comes to the project closure phase now, and we would complete the final report, record the video and summarize the experience learned from this project.

2.4 Risk Management

A risk means that something that will affect the project, product and business negatively may happen during the software development process. We were mainly facing three kinds of risks: project risks, product risks, and business risks. We took the risk factor into account in the early and middle stages of software development, and we managed to solve the problem despite stumbling blocks.

2.5 Quality Management

High-quality software should not only be user-friendly, but also easy for programmers to modify or upgrade.

3. Requirement

3.1Techniques for Finding Requirements

3.1.1 Background Reading

Initially, the iteration was conducted in the form of a flowchart to sort out the handout of group operations to get a global understanding of the elements and steps of the check-

in system. In addition, we searched and studied a large number of existing check-in systems, such as China Southern Airlines, to get some general user requirements, such as seat selection.

3.1.2 Questionnaire Survey

In order to conduct an organized and targeted needs analysis, we decided to conduct a questionnaire survey. The group we focused on was the frequent flyers and had a wide age range. The project was designed to meet the needs of basically all users. Therefore, in the questionnaire, we mentioned "What difficulties do you encounter when checking in on your own?" to collect needs.

3.1.3 Observation

Our system was further optimized by looking at the check-in systems on the market and the users who were using it. The result was that users needed different login methods, a variety of meal options, etc., which were categorized as other requirements.

3.2 Prototype User Interface

Please refer to the appendix 1.

3.3 Detail Description of Requirements

3.3.1 Functional Requirements

a. Login Page

- ✓ Navigated main interface, user selects according to the situation.
- ✓ Allow users to log in by reservation code, name and id, id card.
- ✓ Check the existence of reservation information and display the corresponding prompt.
- ✓ Displays personal information to confirm identity.

b. Seat Selection Page

- ✓ Display the aircraft floor plan to facilitate direct user selection.
- ✓ Seats selected and seats available are displayed in different colors.
- ✓ Allows the user to choose to obtain a paid seat.
- ✓ Allows users to select different types of special seats.

c. Meal Selection Page

- ✓ Provides three choices: default meal, paid special meal and no meal selection.

- ✓ Allows users to select multiple types of meals and beverages separately.
- ✓ Prompt for special meal prices.

d. Payment Page

- ✓ Displays selected options, unit price and total price.
- ✓ Allows users to pay by multiple methods.

e. Print Page

- ✓ Lists all selected options for checking.
- ✓ Displays gate, baggage carousel, and other flight information.
- ✓ User can watch the boarding pass printing animation while waiting.
- ✓ A progress bar is displayed at the top of the screen to provide the user with a time and reference.

3.3.2 Non-Functional Requirements

a. Reliability requirements: The system is fault tolerant, each screen can go back to the previous screen or the main menu to avoid accidental selection or failure, and basic limitations and error checking have been taken into account.
b. Availability requirements: Create a user-friendly, easy-to-use system. Designs such as the "Help" button are designed to create a good user experience.
c. Maintainability requirements: A flexible and extensible design of the system with clear boundary classes, entity classes and control classes for later use in the marketplace.
d. Consistency requirements: Each functional page is displayed in the same interface and in a consistent style.

3.4 Priorities and Estimation of Stories

3.4.1 Priorities of the User Stories

Please refer to the appendix 2.

3.4.2 Product Backlog Evolution

As the system iterates and evolves, the first version of the backlog requires certain modifications and changes to meet various requirements.

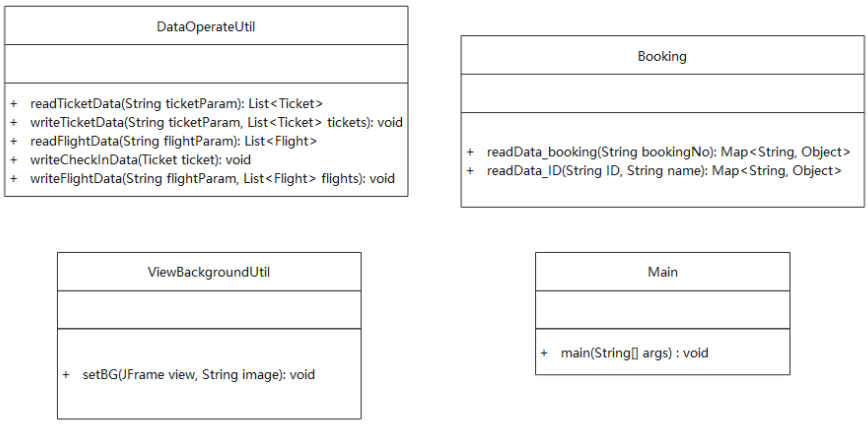
3.4.3 Improvement of Previous Backlog

Please refer to the appendix 3.

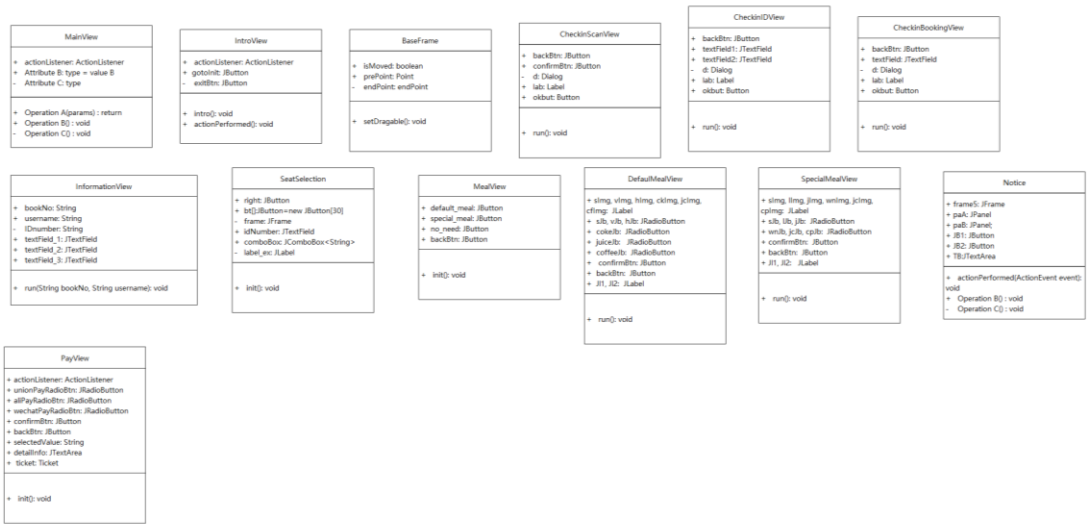
4. Analysis

4.1 Identify Entity, Boundary and Control Classes

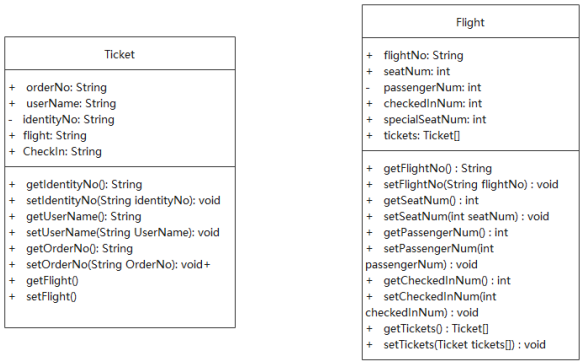
4.1.1 Control Classes



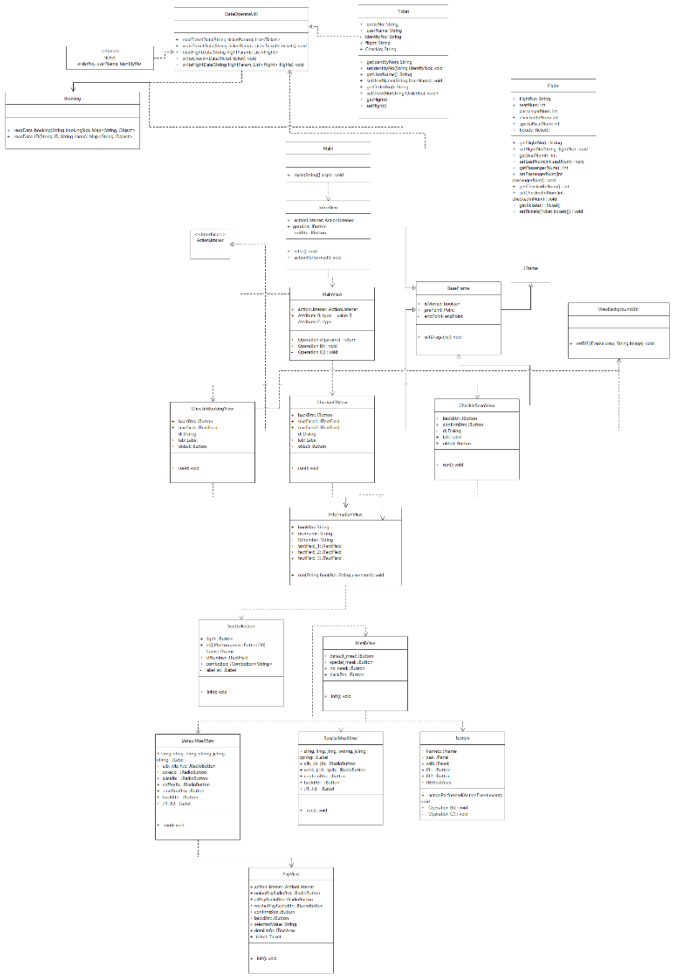
4.1.2 Boundary Classes



4.1.3 Entity Classes



4.2 Conceptual Class Diagram



5. Design

5.1 Reusability and Adaptability

a. User Interface Part Reuse:

We have designed a base structure for our user interface in the class BaseFrame, and each window of our program is designed based on the BaseFrame. By following this design principle, we can unify the user interface style, give users a smooth and consistent experience with the software, and make it easier to add new windows or modify the design of existing UI in the future.

b. Utility Part Reuse:

Besides, we have created a number of utility classes for easier reuse. For example, DataOperateUtil is designed for dealing with data IOs with our txt files, ProgressBarUtil is designed for displaying a progress bar on every window of the software, CustomButton is to generate buttons with a specific style that we designed. All these classes as well as methods in these classes are employed many times in the software, which embodies our design principle of reusability.

c. Adaptation to New Changes:

We have considered about achieving expandability at every aspect of the software, for its convenience to adapt to future changes and new options and features. The software is flexible and extensible. The kiosk can be easily customised for a different airline if needed. it is easy to add new options. Take the seat selection function for example, the software is compatible with airplanes of different types. The difference of seat numbers and seat distribution in the plane as well as the set of first-class seats. It can easily add a new plane model if we want. The meal selection part and extra option part also show our considerations to the software's adapting capability. We deliberately chose our data types and carefully designed our methods, so that the software can easily add additional extra requirements apart from special food types and special seats. We can change the special meal menu and add new food types to the system, and change their prices respectively.

5.2 Design Principles

During the design stage, we followed carefully to the SOLID rule of software design principles.

5.2.1 SRP(Single Liability Principle)

The principle of single responsibility is the guideline to achieve high cohesion and low coupling. We carefully mapped out the different responsibilities of the classes and separated them so that one class was responsible for the corresponding responsibilities

in only one functional area, with high cohesion. When any module changes, functions performed by other modules are not affected, resulting in loose coupling.
For example, we split the user interface presentation and function execution of the software into different classes of different packages so that they perform different responsibilities

5.2.2 OCP (Open-Closed Principle)

The open closed principle means that software entities should be open for extension and closed for modification. That is, software should be as extensible as possible without modifying the original code.

For example, our CustomButton class is an extension of the JButton class. We add the airplane seat button, the customized button, and the changes before and after the button selection. These are extensions of the existing JButton class, but we do not change the original function of the button.

5.2.3 DRY(Don't Repeat Yourself)

When we program, we try not to duplicate our code. When a lot of similar functions in the program are copied and cut from the original code and changed slightly, if the original function has problems and needs to be changed, it will be difficult to adjust the code. When some variables change, if we don't follow the Don't DRY principle, we need to change a lot.

To handle duplicate methods, we put them in a new class. We can call these methods when we need them. This way, when we need to change these methods, we only need to change them once. For example, we often read and write files, so we write an IO class that contains methods for reading and writing files. We call this method when we need it, and we only need to adjust the code in the method once in the future when the operation changes. .

5.2.4 DIP (Dependency Inversion Principle)

The Dependency Inversion Principle states that high level modules should not depend on low level modules; instead, both should depend on abstractions. Abstractions should not depend on details; instead, details should depend on abstractions Reduce coupling between classes
The reason we follow the DIP principle is that it can improve system stability and improve system maintainability and scalability.

5.2.5 ISP (Interface Segregation Principle)

The interface isolation principle means that an interface of a certain type should contain only a small number of methods so that clients are not forced to rely on methods they do not use. In our design process, we kept the interfaces as small as possible, and we only put methods in one interface when they were highly correlated.

5.2.6 LSP (Liskov Substitution Principle)

The Liskov Substitution Principle states that Inheritance should ensure that any property proved about supertype objects also holds for subtype objects. In other words, A subclass can extend the functionality of its parent class but cannot change the functionality of its parent class. It can be separated into four parts:

a. A subclass can implement an abstract method of the parent class, but cannot override a non-abstract method of the parent class.
b. Subclasses can add their own new methods.
c. When a subclass's method overrides a parent class's method, the method's preconditions (that is, the method's input/input parameters) are looser than the parent method's input parameters.
d. When a subclass's method overrides a parent class's method, the method's postcondition (that is, the method's output/return value) is stricter or equal to that of the parent class.
e. We carefully considered the inheritance relationships in our software to make sure they didn't break the Liskov Substitution Principle, and none of our inherited classes changed the functionality of the superclass, only added new functionality to it.

6. Implementation

6.1 Configuration Management

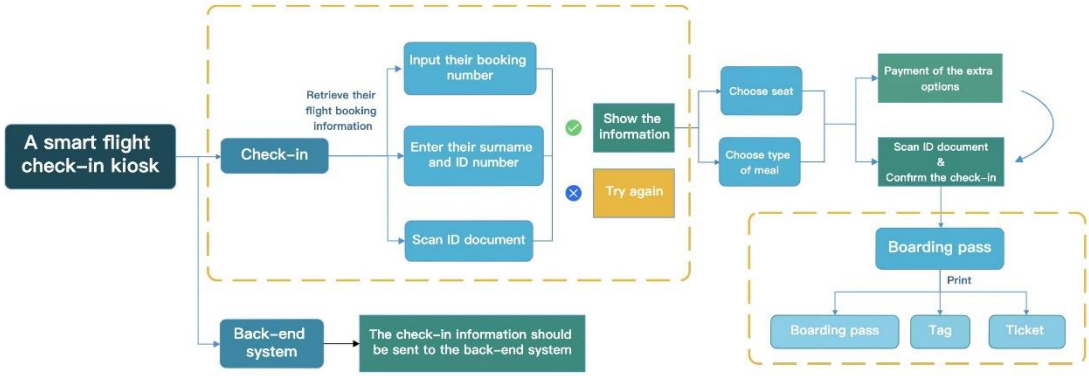
The essence of software development is to cater to users' requirements, so in order to respond to real-time changes in requirements, we applied the QMHub tool as a remote serve to implement parallel development. Most of our main functions were completed in the second iteration, such as the change of the type of special meals, the optimization of the seat selection interface, which doesn’t allow passengers select the seat already selected by other passengers in front, and one passenger can select multiple flights, the addition of card number and password in the payment interface, and the optimization of the close interface, help and home buttons and relative interfaces.

During the process of software development, we also need to pay attention to the software version and change management. There are two main reasons: First, we need to ensure that the members work in parallel to ensure that the work is not related to each other, so as to prevent the content from crossing and causing confusion in the code. Second, we always keep the previous working version, because as requirements change, features and functions need to be adjusted at any time, which may lead to some irreversible conflicts, including some interface optimizations that cannot be unified across different computer operating systems.

We created the main branch to store the successfully debug able code, and added new functions in the branch. If the merger was successful, the branch would be deleted and the source code would be directly covered. However, since there was little correlation between the codes of each person and they did not affect each other, most of them were uploaded uniformly after the completion of integration and modulation.

6.2 Iteration Plan

The first iteration was mainly about meeting and discussion, determining main functions and unifying interface background. The following figure is the record of meeting and discussion in the first iteration.



The second iteration begins to implement functionality according to the division of labor. The main functions are: Logo, login screen (the identity information input and determine), choose flight (one can select multiple flights), meal selection (special meals on the type of change), seat selection (tips for special seats and ban of choosing seats bought by other passengers), payment interface (card number and password), visual confirmation interface optimization, and add help and Home buttons to the close interface.

The rest of the iteration process is mainly to optimize details, such as providing Alipay QR code pictures and other services that are more consistent with the actual check-in system, and improving the visual fluency of page skipping.

6.3 Implement Strategy

The implementation strategy we adopted was divided into two parts. The first is iterative development, with each iteration producing a working application. Test on this basis and release software for customer use after passing alpha testing. The other is to divide the implementation of each iteration into a number of non-repeating tasks. By working in parallel, we can improve the efficiency of the group. When each team member has completed their own tasks, the group leader will integrate and test to ensure the correctness and reliability of the integrated code.

After each iteration, we would review and reflect to find the areas needing improvement in the current iteration and add the areas needing improvement into the requirements of the next iteration. The advantage of this is that the Agile approach takes full advantage of the rapid-response nature of our approach. After each iteration, by asking customers and reviewing ourselves, we find areas where we can optimize our software to meet the changing needs of our customers and the market in real time.

6.4 Integration of Build Plan

You can refer to the Journal Page of our QMHub website of our team's iteration plan. Here is the link: https://hub.qmplus.qmul.ac.uk/artefact/blog/view/index.php?id=1009831

7. Testing

7.1 Testing Strategy

a. 60% of the tests will be automated.
   - The remainder will be manual.
   - Ensure that the user interface is correct.
   - Each subject use case is tested for its normal flow and behaviour and also two alternative flows.
   - Ensure that we cover incorrect user input or operations.

b. Success criteria -95% of the test cases passed.
c. No high priority defects unresolved.

7.2 Testing Techniques

a. Black-box testing
   We use black-box testing for both unit testing and integration testing. To test the

functional requirements of our check-in system, black-box testing attempt to find errors, miss functions, incorrect functions, interface errors and behaviour errors.

- ✓ Partition testing
  We use partition testing for both component testing and system testing. When using our system, users can have multiple types of input or choices at each step. To test whether the system is robust enough in these cases, partition testing is adopted. For example, our system only accepts passengers with correct booking number or name and ID to check in.
- ✓ Regression testing
  We use regression testing for integration testing. At each stage of iteration, we create test cases to test the functionality of the build.

b. White-box testing
We make sure that at each step, each class is fully unit tested using white-box testing. We use white-box testing to test the internal program logic to ensure that internal operations are performed according to specifications. For basis path testing, we apply it for core functions, such as selecting seat and meal.

7.3 Test Case

Test case for check in with booking number

- ✓ Test case for BookingNum (with correct input)
  -Input: valid booking number
  -Result: A pop-up shows that you have successfully logged in.
  -Conditions: No conditions exist on this test.
- ✓ Test case for BookingNum (with incorrect input)
  -Input: invalid booking number which length is less than six.
  -Result: A pop-up shows that your booking number is invalid.
  -Conditions: The password should satisfy the length criteria.
- ✓ Test case for BookingNum (with incorrect input)
  -Input: invalid booking number which length is more than six.
  -Result: A pop-up shows that your booking number is invalid.
  -Conditions: The password should satisfy the length criteria.
- ✓ Test case for BookingNum (with incorrect input)
  -Input: invalid booking number which has characters other than digits.
  -Result: A pop-up shows that your booking number is invalid.
  -Conditions: The password should satisfy the character criteria.
- ✓ Test case for BookingNum (with incorrect input)
  -Input: invalid booking number which only has letters other than digits.
  -Result: A pop-up shows that your booking number is invalid.
  -Conditions: The password should satisfy the character criteria.

Note: Test cases for check in with name and ID number and test cases for Paypal card number and password verification are shown in appendix.

7.4 Test Matrix

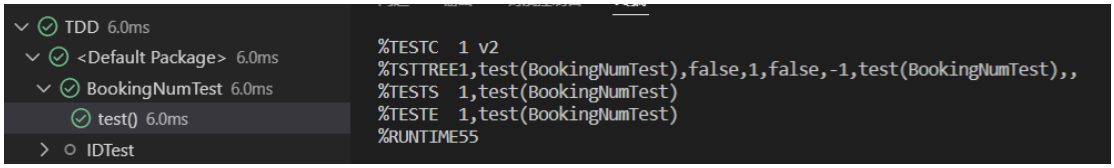
Test matrix for booking number format verification.

Test Case: File Open	#Test Description	Test Cases	Samples Pass/ Fail	No. of Bugs	Bug#	Comments
N/A	Booking number format check	Setup	-	-	-	Added to system for testing purposes
1.1	Check correct booking number (123456)	1.1	P	0	0	Correct behaviour observed
1.2	Check the booking number (123) length less than criteria	1.2	P	0	0	Correct behaviour observed
1.3	Check the booking number (1234567) length more than criteria	1.3	P	0	0	Correct behaviour observed
1.4	Check the booking number (98#%bb) with other characters	1.4	P	0	0	Correct behaviour observed
1.5	Check the booking number (A) with no digits	1.5	P	0	0	Correct behaviour observed

Note: Test matrix for name, ID number, Paypal card number and password verification are shown in appendix.

7.5 TDD

We use Junit in the progress of TDD, which is a simple unit-testing framework for supporting TDD. During this process, we write the code of the class to meet the specification until all tests pass. Here is the test of BookingNumTest:



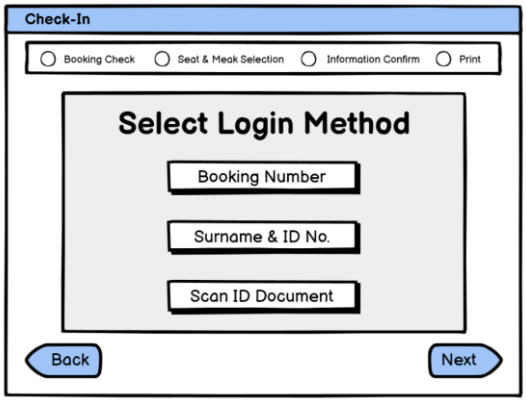
8. Conclusion

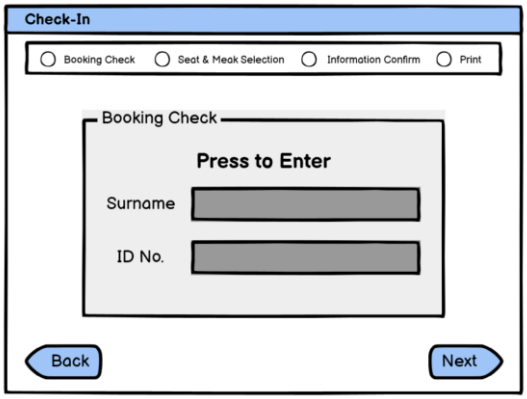
After five iterations of development, our team completed the online fitness system and related documents on schedule. Through the analysis of user needs in each iteration, we found a variety of system functions that met the actual needs, such as adding help and home buttons in addition to the basic functions. Through prototyping, we present our ideas in a visual form, giving users a clearer understanding of our system and its functions. Through design and analysis, we set up a good start for our system and laid a stable foundation for the program. Through testing, we ensure that the program is robust to any form of input. Finally, under the clear division of labor and good cooperation, we implemented the system well. Through this practical experience, we have experienced the advantages and efficiency of the Agile software development process, and have more understanding of the theory of software engineering in all aspects.

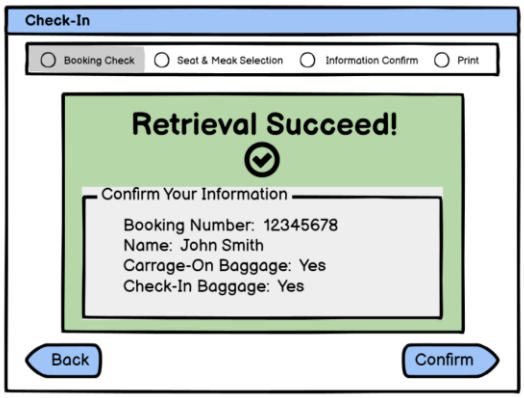
Appendix

1. Prototype User Interface

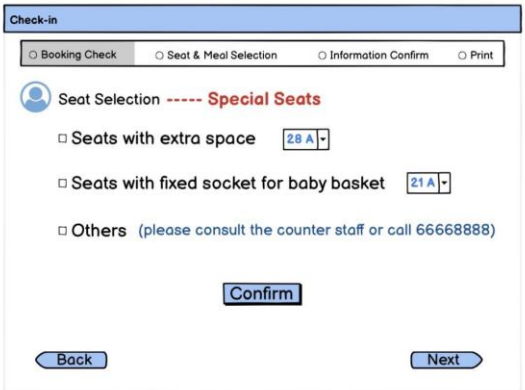
a. Login Page Prototype

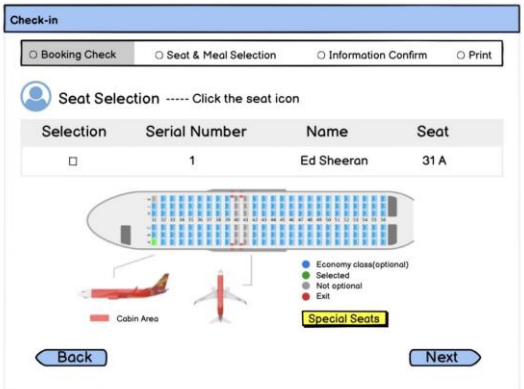




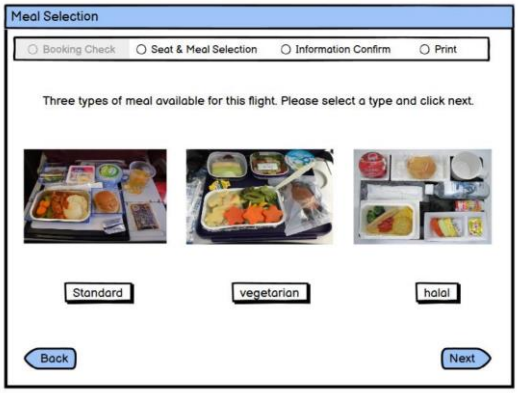


b. Seat Selection Page Prototype



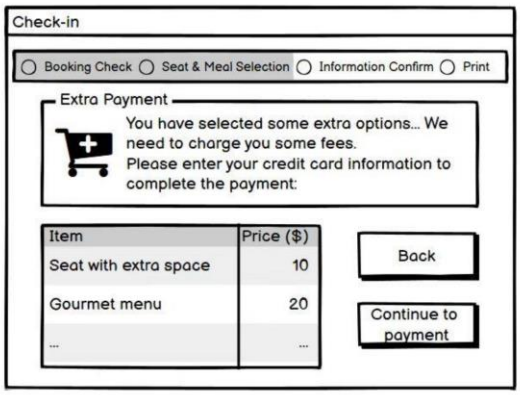


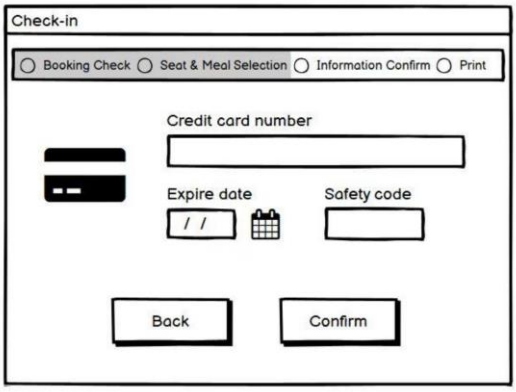
c. Meal Selection Page Prototype





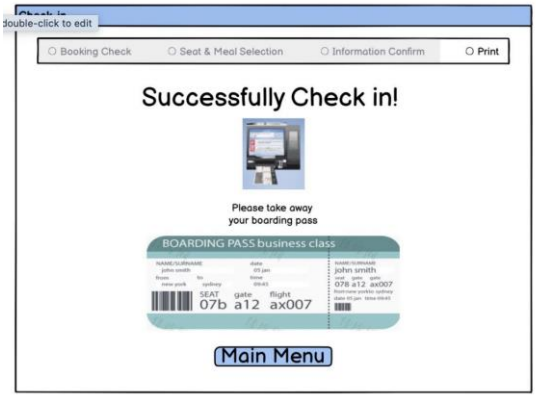
d. Payment Page Prototype





e. Print Page Prototype





2. Priorities of the User Stories

Story ID	Story Name	Priority
1	Back-End System Receives Passenger List	Very High
2	Choose and Print Boarding Pass	Very High
3	Seat Choosing Page	Very High
4	Correctly Choose Seats	Very High

5	Food Choosing	Very High
6	Save Choices	Very High
7	Three Ways of Identity Authentication	Very High
8	Retrieve Booking Information	Very High
9	Navigation Page	Very High
10	Payment for Extra Options	High
11	Special Choices	High
12	Display Summary Information after Successful Retrieval	Medium
13	Display Summary Information	Medium
14	Cancel Operations	Low
15	Clear Error Indication	Low
16	Progress Bar	Very Low
17	Adjust Font Sizes and Bigger Buttons	Very Low
18	Switch Languages	Very Low

3. Improvement of Previous Backlog

Story ID	Story Name	Priority
1	Login Method	Very High
2	Print Boarding Pass	Very High
3	Seat Selection	Very High
4	Meal Selection	Very High
5	Retrieve Booking Information	High
6	Back-End Connection	High
7	Special Choices	High
8	Display Confirmation Information	Medium
9	Navigation Page	Medium
10	Seat Status	Medium
11	Save Choices	Medium
12	Payment	Medium
13	Display Summary Information	Medium
14	Cancel Operations	Low
15	Error Indication	Low
16	Progress Bar	Very Low
17	Help Button	Very Low

4. Test Case and Test Matric

a. Test Case

- Test case for check in with name

- ✓ Test case for Name (with correct input)
  -Input: valid name.
  -Result: A pop-up shows that you have successfully logged in.
  -Conditions: No conditions exist on this test.
- ✓ Test case for Name (with incorrect input)
  -Input: invalid name which has characters other than letters.
  -Result: A pop-up shows that your name format is invalid.
  -Conditions: The name should satisfy the character criteria.
- ✓ Test case for Name (with incorrect input)
  -Input: invalid name which has characters other than letters.
  -Result: A pop-up shows that your name format is invalid.
  -Conditions: The name should satisfy the character criteria.

- Test case for check in with ID number
- ✓ Test case for ID (with correct input)
  -Input: valid ID number.
  -Result: A pop-up shows that you have successfully logged in.
  -Conditions: No conditions exist on this test.
- ✓ Test case for ID (with incorrect input)
  -Input: invalid ID number which length is less than six.
  -Result: A pop-up shows that your ID number is invalid.
  -Conditions: The ID number should satisfy the length criteria.
- ✓ Test case for ID (with incorrect input)
  -Input: invalid ID number which length is more than six.
  -Result: A pop-up shows that your ID number is invalid.
  -Conditions: The ID number should satisfy the length criteria.
- ✓ Test case for ID (with incorrect input)
  -Input: invalid ID number which has other characters.
  -Result: A pop-up shows that your ID number is invalid.
  -Conditions: The ID number should satisfy the character criteria.
- ✓ Test case for ID (with incorrect input)
  -Input: invalid ID number which doesn't have three upper letters.
  -Result: A pop-up shows that your ID number is invalid.
  -Conditions: The ID number should satisfy the character criteria.

- Test case for check in with Paypal card number and password
- ✓ Test case for Paypal (with correct input)
  -Input: valid Paypal card number and password
  -Result: A pop-up shows that you have paid successfully.
  -Conditions: No conditions exist on this test.
- ✓ Test case for Paypal (with incorrect input)
  -Input: invalid card number which length is less than twelve.
  invalid password which length is more than fifteen.
  -Result: A pop-up shows that your card number and password are invalid.

-Conditions: The card number and password should satisfy the length criteria.

- ✓ Test case for Paypal (with incorrect input)

-Input: invalid card number which length is more than twelve.
invalid password which length is less than eight.
-Result: A pop-up shows that your card number and password are invalid.
-Conditions: The card number and password should satisfy the length criteria.

- ✓ Test case for Paypal (with incorrect input)

-Input: invalid card number which has characters other than digits.
invalid password which only has digits other than letters.
-Result: A pop-up shows that your card number and password are invalid.
-Conditions: The card number and password should satisfy the character criteria.

- ✓ Test case for Paypal (with incorrect input)

-Input: invalid booking number which only has letters other than digits.
valid Paypal card number
-Result: A pop-up shows that your password is invalid.
-Conditions: The password should satisfy the character criteria.

- ✓ Test case for Paypal (with incorrect input)

-Input: invalid password which only has other characters.
valid Paypal card number
-Result: A pop-up shows that your password is invalid.
-Conditions: The password should satisfy the character criteria.

b. Test Matric

Test matrix for name format verification.

Test Case: File Open	#Test Description	Test Cases	Samples Pass/ Fail	No. of Bugs	Bug#	Comments
N/A	Name format check	Setup	-	-	-	Added to system for testing purposes
1.1	Check correct name (tom)	1.1	P	0	0	Correct behaviour observed
1.2	Check the name (123) with other characters	1.2	P	0	0	Correct behaviour observed
1.3	Check the name (to&m) with other characters	1.3	P	0	0	Correct behaviour observed

Test matrix for ID number format verification.

Test Case: File Open	#Test Description	Test Cases	Samples Pass/ Fail	No. of Bugs	Bug#	Comments
N/A	ID number format check	Setup	-	-	-	Added to system for testing purposes
1.1	Check correct ID number (ABC123)	1.1	P	0	0	Correct behaviour observed
1.2	Check the ID number (abc123) length less than criteria	1.2	P	0	0	Correct behaviour observed
1.3	Check the ID number (abc1233) length more than criteria	1.3	P	0	0	Correct behaviour observed
1.4	Check the ID number (AAB123&&) with other characters	1.4	P	0	0	Correct behaviour observed
1.5	Check the ID number (111111) with no three upper letters.	1.5	P	0	0	Correct behaviour observed

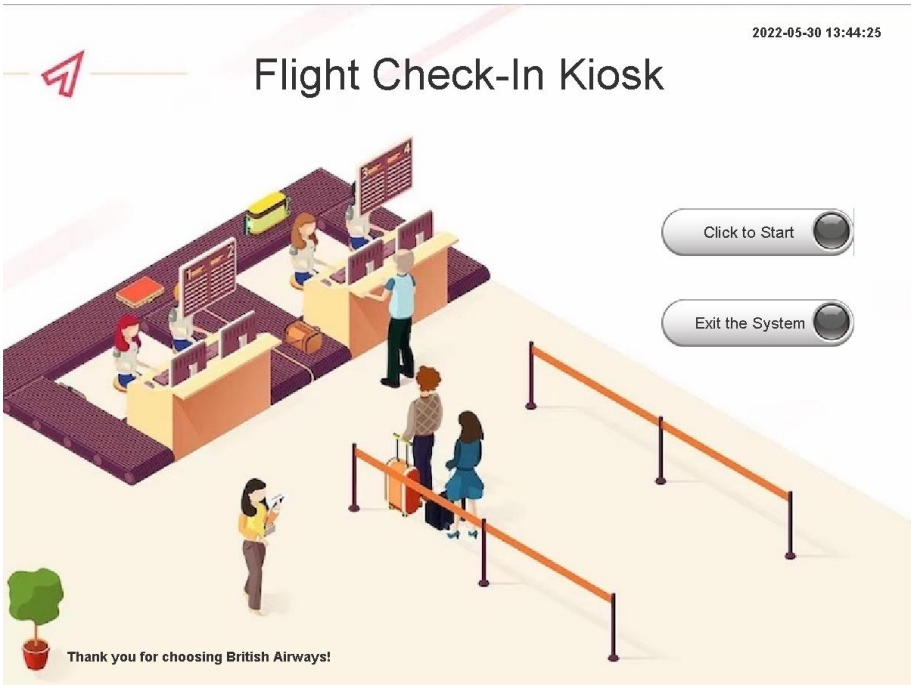
Test matrix for Paypal format verification.

Test Case: File Open	#Test Description	Test Cases	Samples Pass/ Fail	No. of Bugs	Bug#	Comments
N/A	Booking Paypal card number and password format check	Setup	-	-	-	Added to system for testing purposes
1.1	Check correct	1.1	P	0	0	Correct

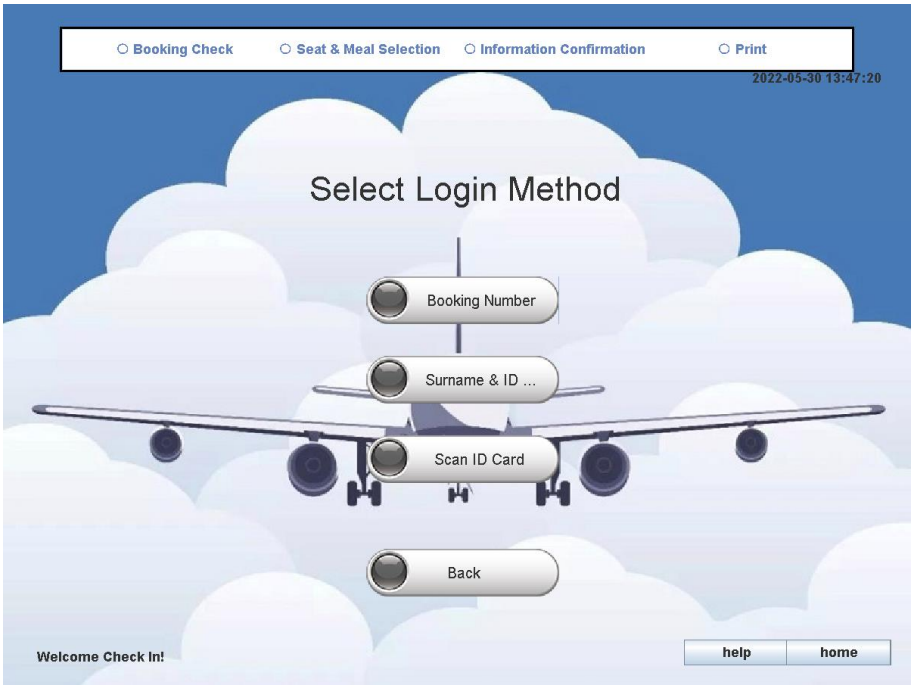
	card number and password (123456789098 ,123456abc)					behaviour observed
1.2	Check the card number length less than criteria and password length more than criteria.	1.2	P	0	0	Correct behaviour observed
1.3	Check the card number length more than criteria and password length less than criteria.	1.3	P	0	0	Correct behaviour observed
1.4	Check the card number with other characters and password only has digits.	1.4	P	0	0	Correct behaviour observed
1.5	Check correct card number and password only has letters.	1.5	P	0	0	Correct behaviour observed
1.6	Check correct card number and password with other characters.	1.6	P	0	0	Correct behaviour observed

5. Main Screenshots of the System

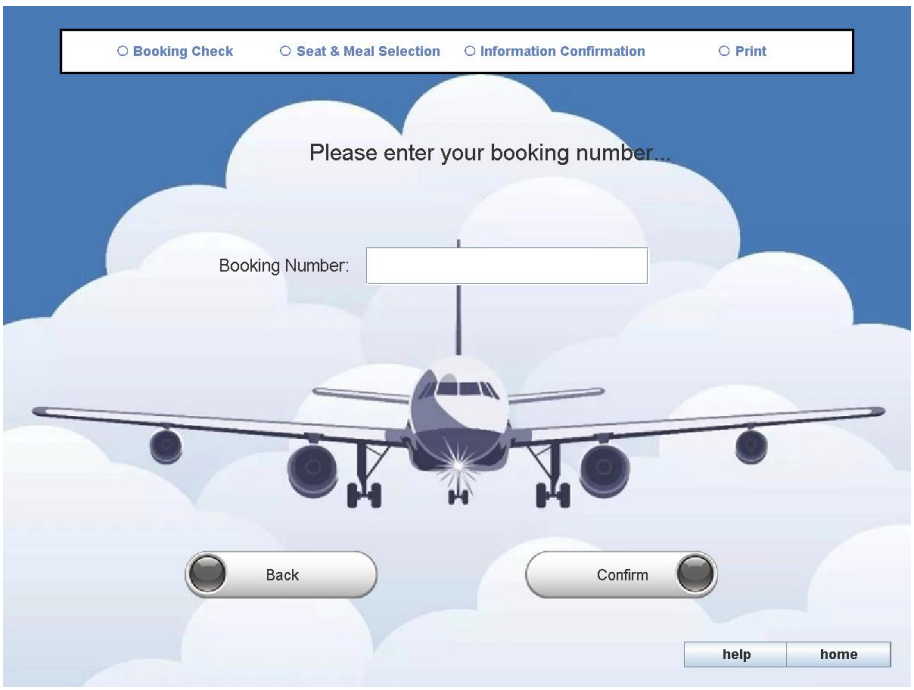
a. Intro Page



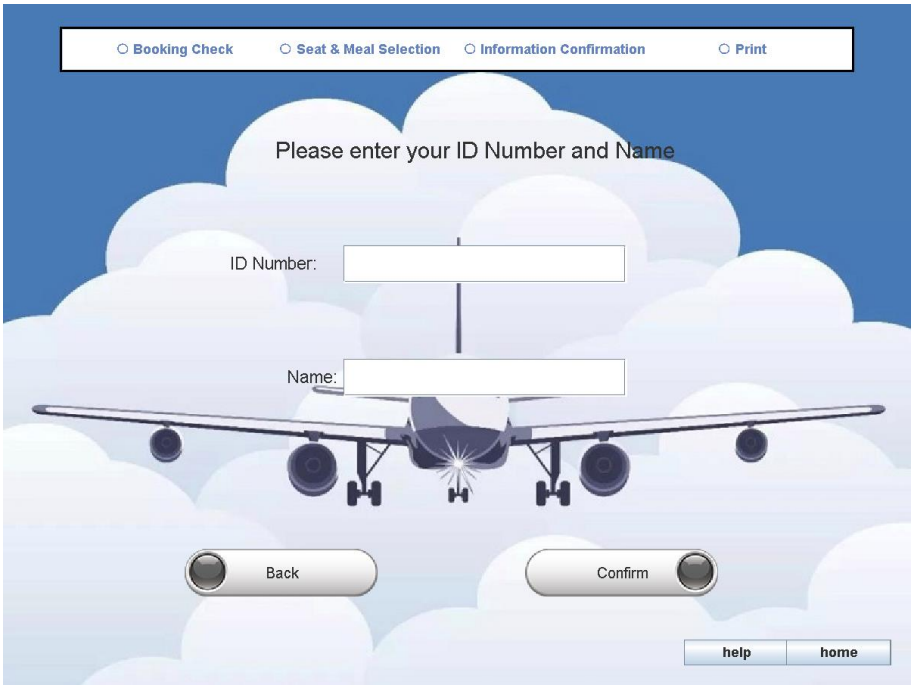
b. Check-In Page (Three Method)



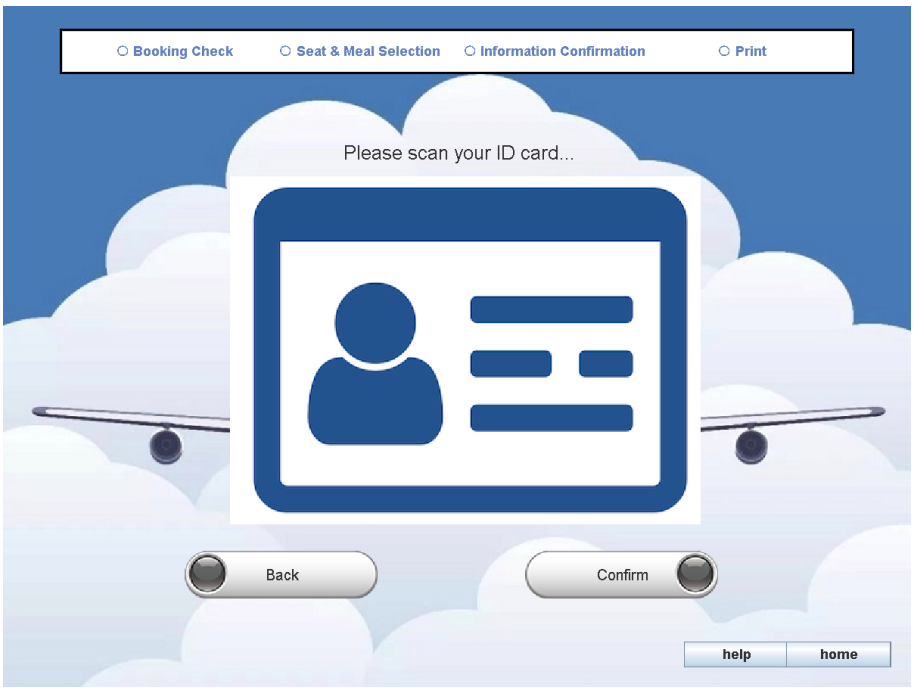
Navigation



Booking Number

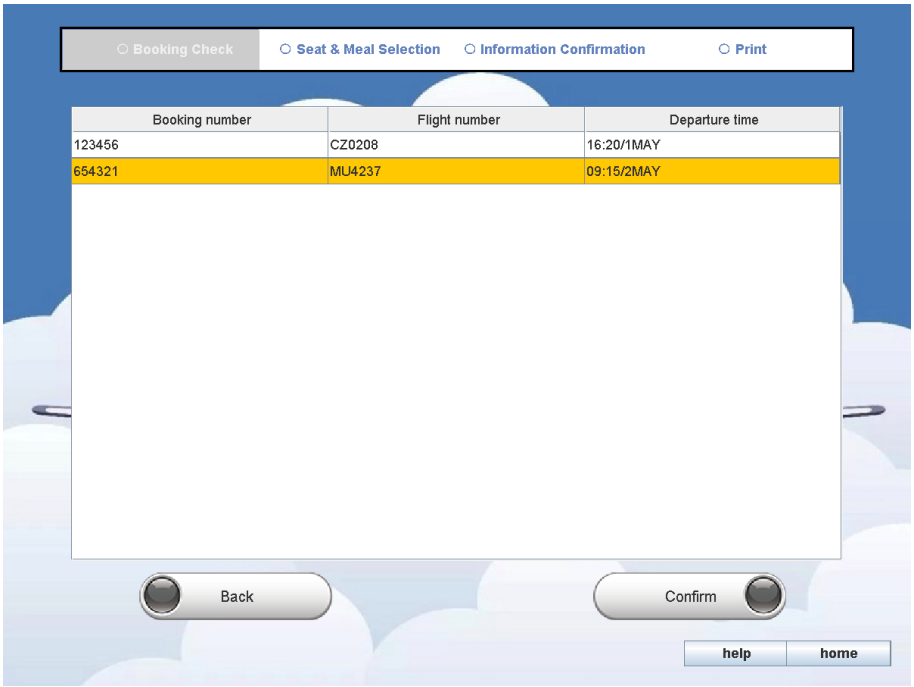


ID Number and Name

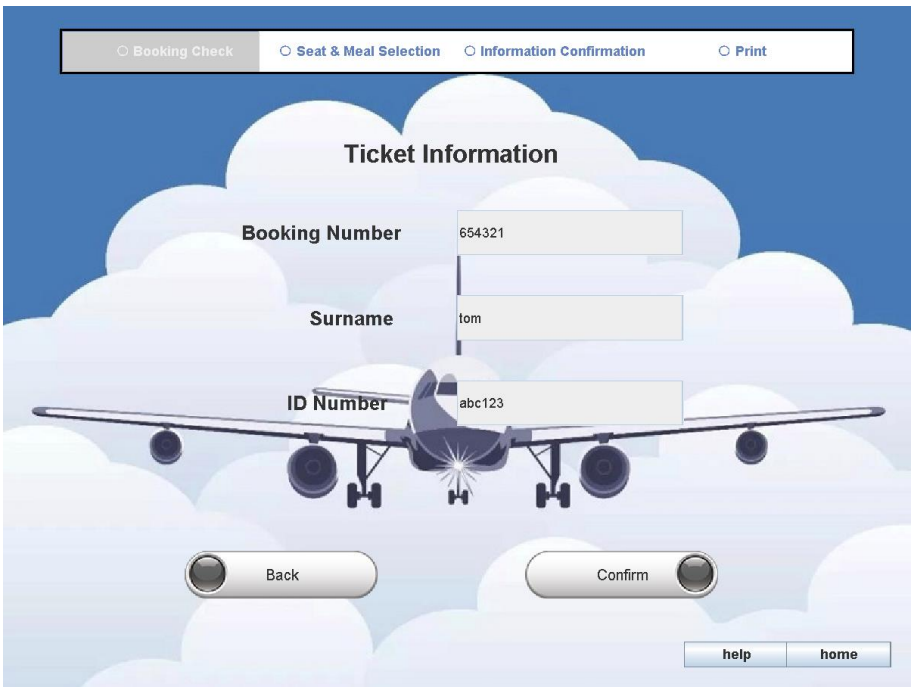


ID Card

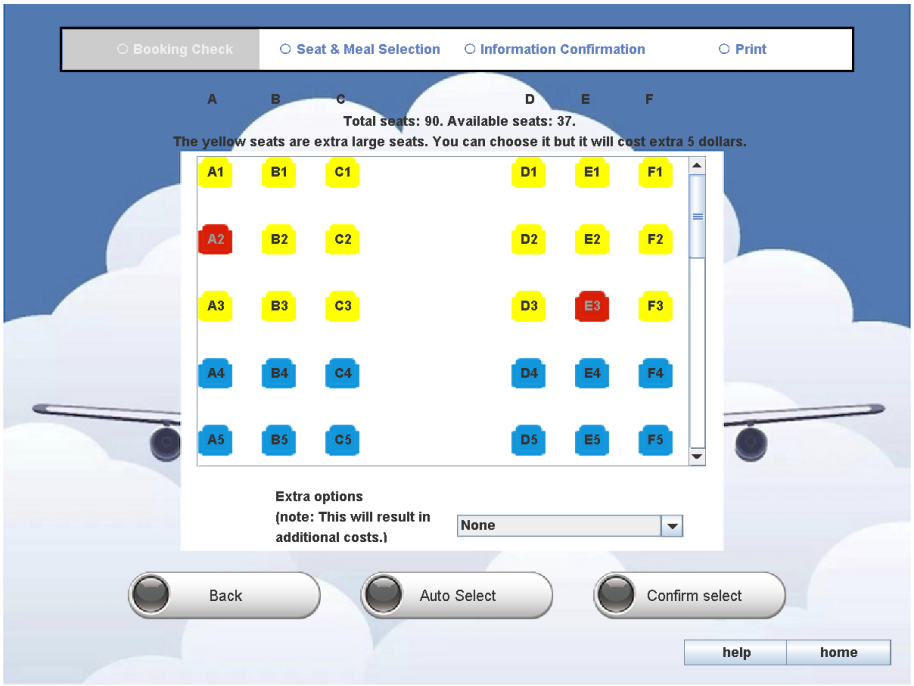
c. Ticket Selection



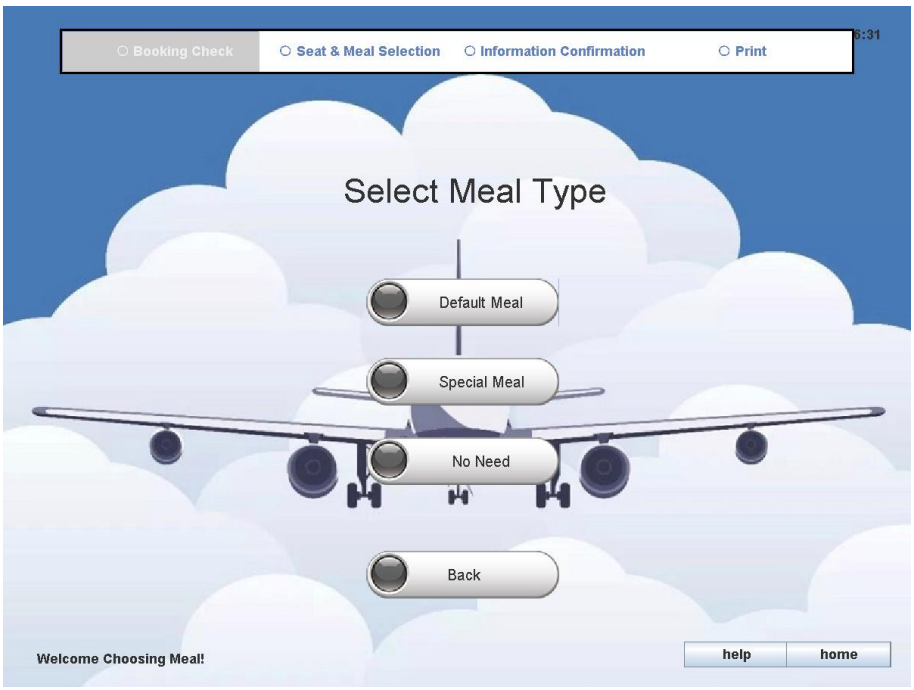
d. Information Retrieval



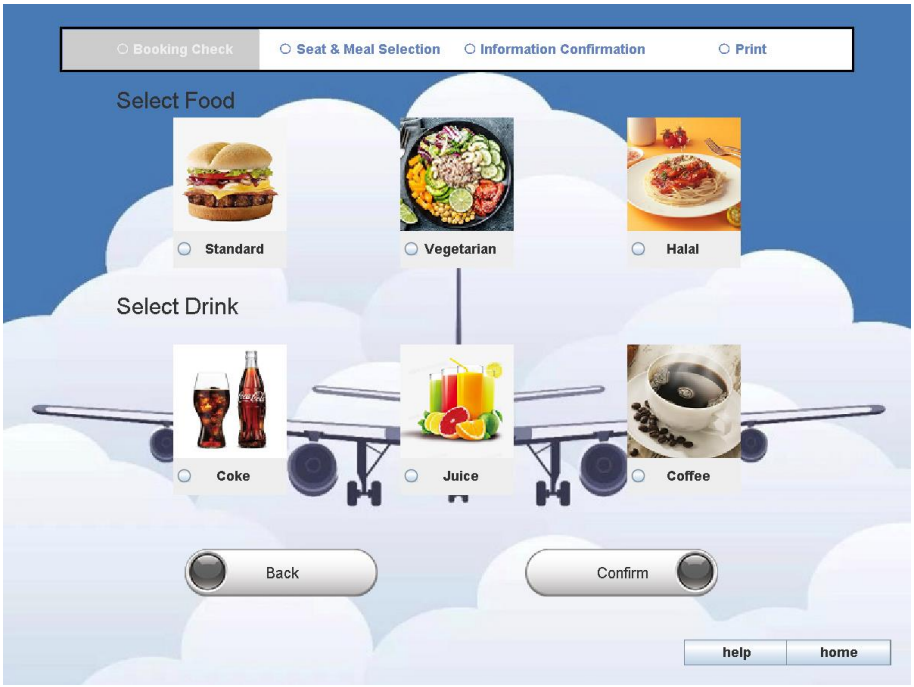
e. Seat Selection



f. Meal Selection



Navigation

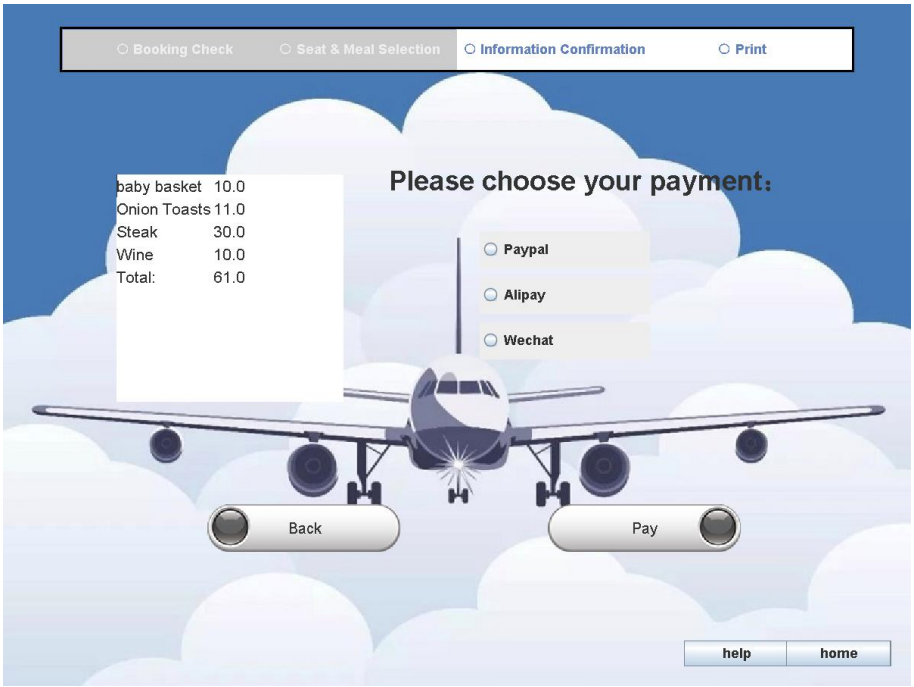


Default

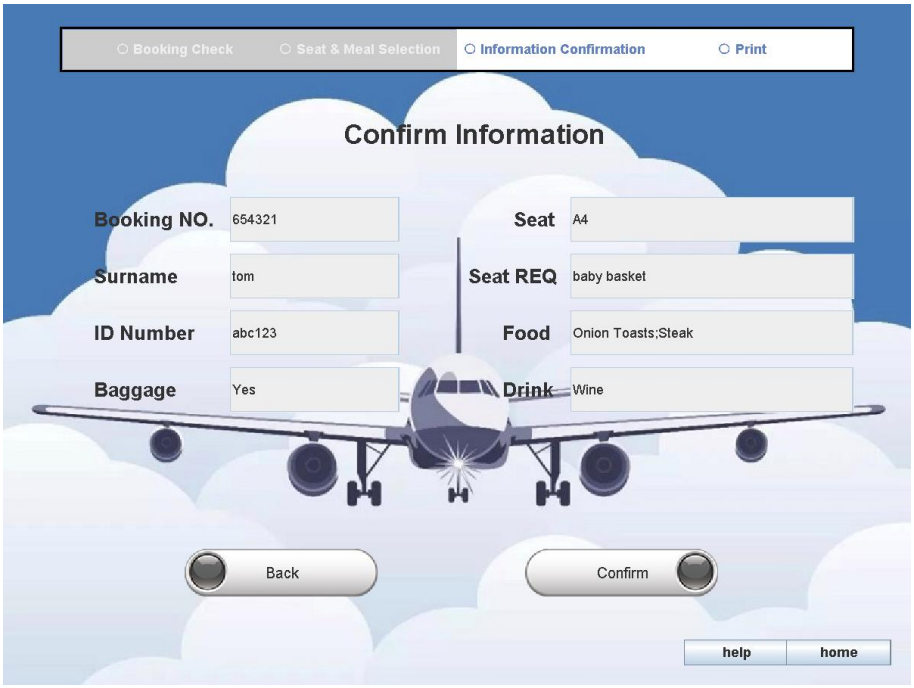


Gourmet Menu

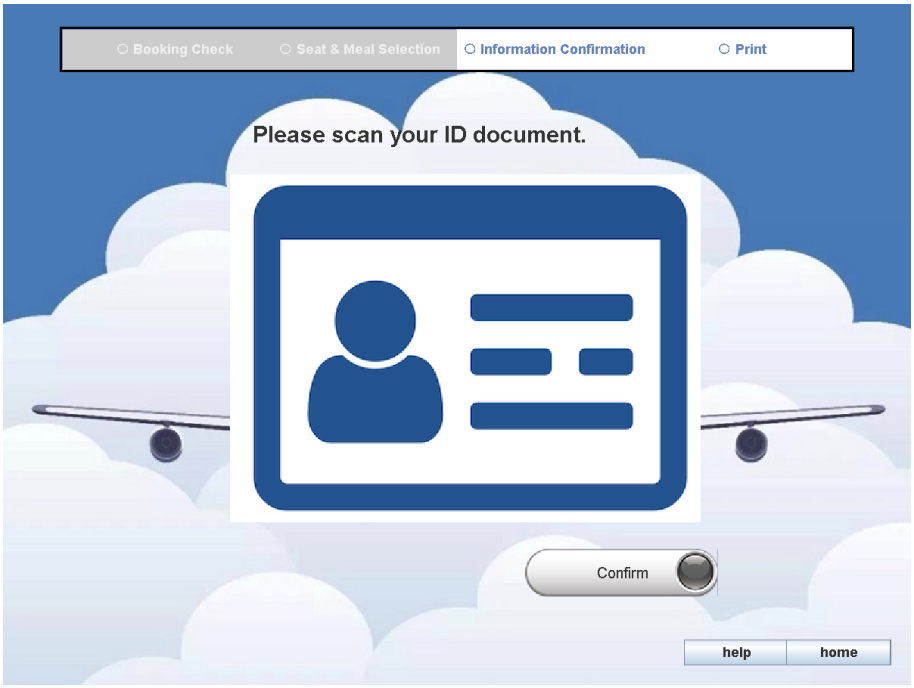
g. Payment



h. Information Confirmation



i. Print Page



Scan ID



Print Boarding Pass

6. Participation and Achievement

NAME	*PREPARATION & CODE*	*REPORT*
Qian Ma	1. During the project preparation process, I was responsible for part of product backlog and prototype. 2. In the code part, I was responsible for main interface, three types of login screen windows, printing interface, interface for scanning ID and ticket display interface, etc. Added "Home" and "HELP" function. Also I helped interface optimization and conduct operational tests on Mac OS.	Write the operation flow of login-related functions.
Wenxin Ning	1. During the project preparation process, I was responsible for part of product backlog and prototype. 2. In the code part, I was responsible for information display page and information confirmation page, and the improvement of the payment function, special meal order function, data file reading and writing function.	In the report part, I was responsible for testing section and part of TDD.
Qianrui Teng	1. During the project preparation process,	Responsible for writing

	I was responsible for part of product backlog and prototype. 2. In the code part, responsible for writing seat selection, extra payment. Added the data IO methods for ticket.txt and flight.txt, added the extra seat option for customers with special needs. Also, helped interface optimization.	the analysis and design of the report.
Quan Yuan	1. During the project preparation process, I was responsible for part of product backlog and prototype. 2. In the code part, I was responsible for writing three meal selection interfaces and extra payment interface. Subsequently, I was in charge of the code debugging and interface optimization of this part.	Responsible for writing the management and implementation of the report.
Zitang Zhou	1. During the project preparation process, I was responsible for part of the product backlog and prototype. 2. In the code part, I organized the software structure file and implemented the function of checking-in in three ways and the reading and writing methods. I also integrated all members' code and deal with code conflicts.	In the report part, I completed the user manual and helped with TDD.
Sijie Zou	1. During the project preparation process, I was responsible for part of the product backlog and prototype. 2. In the code part, I wrote intro page, default and special meal selection page, information confirmation page and process bar function. Also, I helped some part of interface optimization and code debugging.	I was responsible for requirement part and appendix. I completed the integration and layout of the report. Also, I helped user manual and improved final backlog.

7. References

[1] What is scrum? On Scrum. (2018, May) Retrieved May 15th, 2018, 25 from https://zhuanlan.zhihu.com/p/49048186

[2] Is agile project management OK?. (2019, July) Retrieved July 9th, 2019, from https://www.zhihu.com/question/265968122/answer/878124580

[3] Deep interpretation - TDD (Test Driven Development). (2019, April). Retrieved April 9th, 2019, from https://www.jianshu.com/p/62f16cd4fef3